

5. ML Pipeline과 Tuning 소개와 실습

Spark ML 모델 튜닝 소개

타이타닉 승객 생존 예측 모델을 ML Pipeline과 Tuning으로
빌딩

PMML 소개

Contents

1. Spark ML 모델 튜닝
2. ML Pipeline 기반 머신러닝 모델 만들기
3. 범용 머신러닝 모델 파일포맷: PMML
4. 마무리



Spark ML 모델 튜닝

Spark ML이 제공하는 모델 튜닝에 대해 배워보자

◆ Spark ML 모델 튜닝 (ML Tuning)

❖ 최적의 하이퍼 파라미터 선택

- 최적의 모델 혹은 모델의 파라미터를 찾는 것이 아주 중요
- 하나씩 테스트해보는 것 **vs.** 다수를 동시 테스트하는 것
- 모델 선택의 중요한 부분은 테스트 방법
 - 교차 검증(Cross Validation)과 홀드 아웃 (Train-Validation Split) 테스트 방법을 지원
- 보통 ML Pipeline과 같이 사용함

◆ Spark ML 모델 테스트

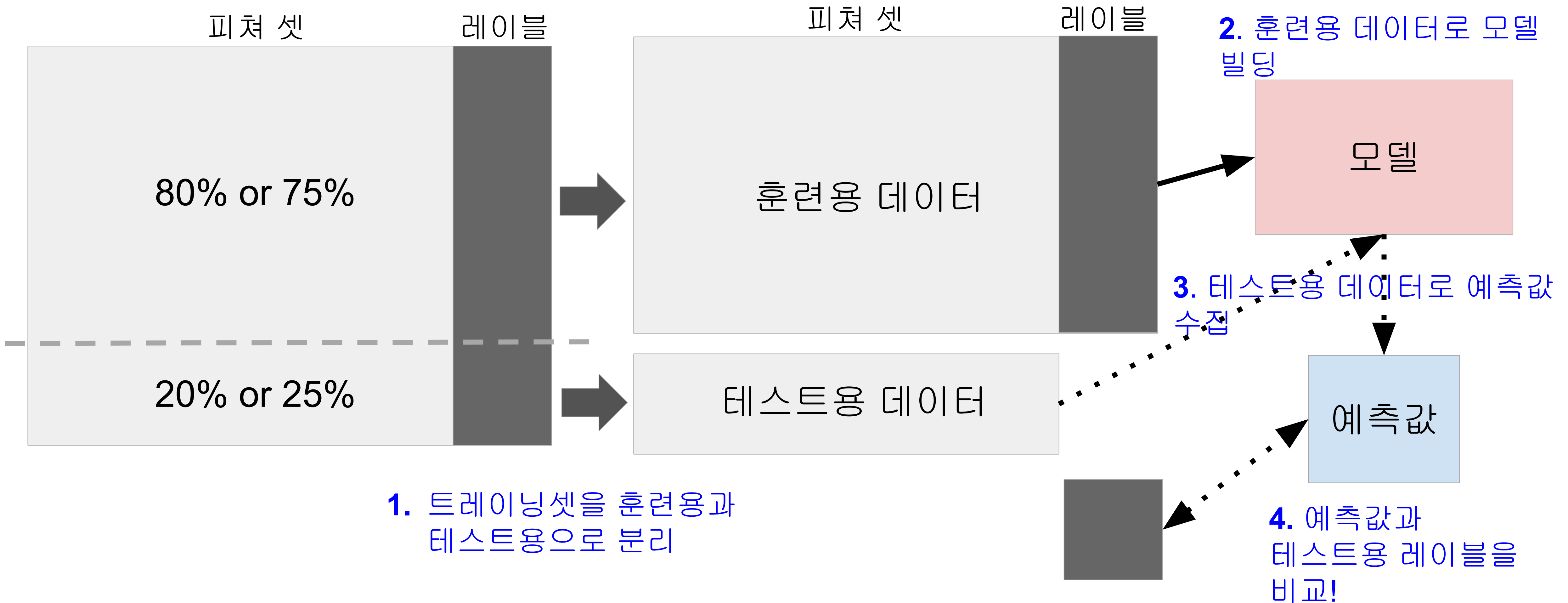
❖ 모델 테스트 방법: 크게 2가지가 존재

- 교차 검증 (Cross Validation): CrossValidator
- 훈련/테스트셋 나누기 (Train-validation split): TrainValidationSplit

◆ Spark ML 모델 테스트

❖ 훈련용과 테스트용 데이터 기반 테스트 (Train & Validation Split)

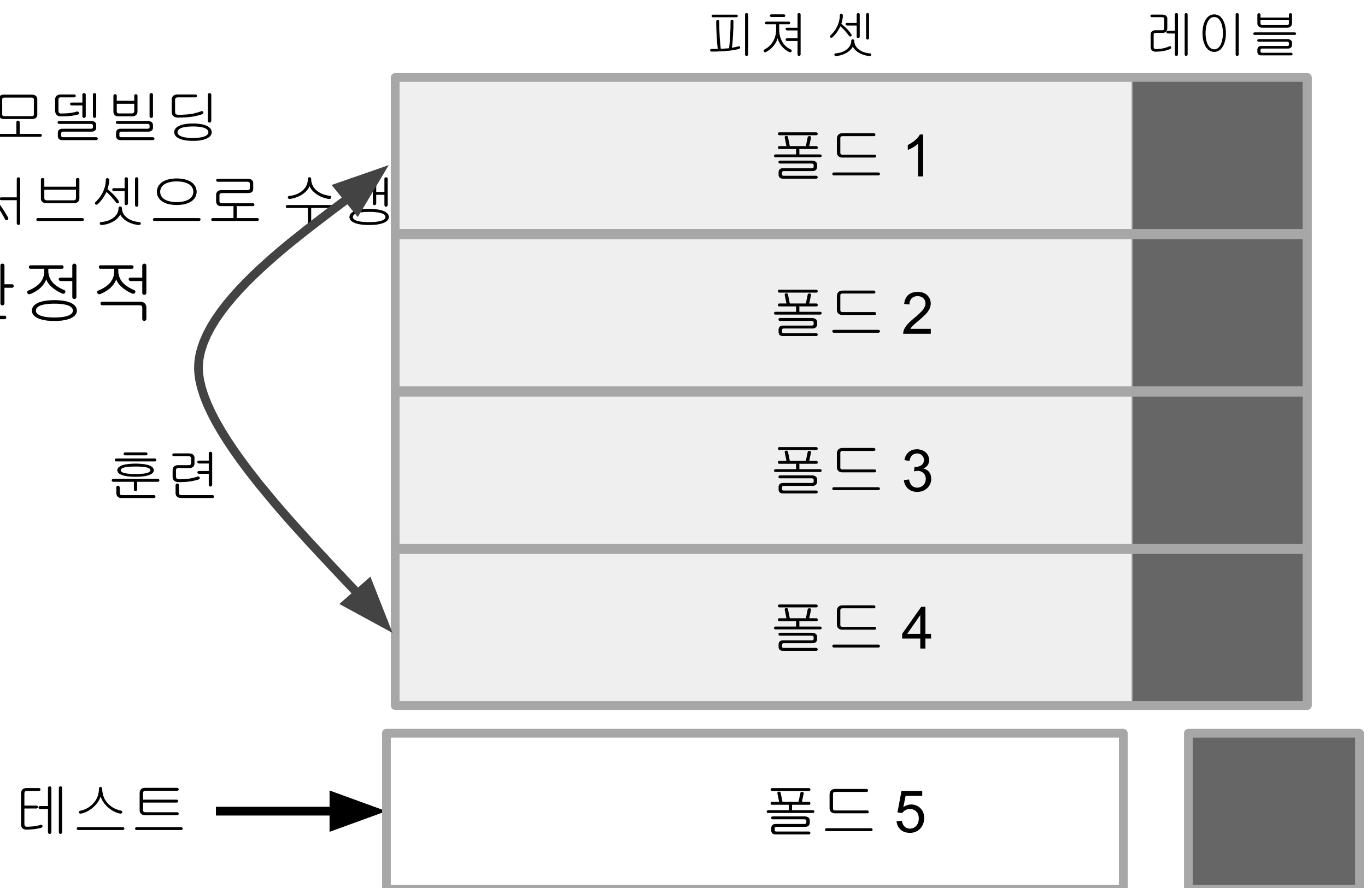
- 홀드아웃 (Holdout) 테스트라고 하기도 함



◆ Spark ML 모델 테스트

❖ 교차분석 테스트 (Cross Validation)

- K-Fold 테스트라고 부르기도 함
- 트레이닝 셋을 K개의 서브셋으로 나눠 총 K번의 훈련을 반복
 - i번째 훈련을 할 때는 아래를 반복
 - i번째 서브셋을 빼고 훈련을 해서 모델빌딩
 - 만들어진 모델의 테스트는 i번째 서브셋으로 수행
- 홀드아웃 테스트보다 훨씬 더 안정적
 - 오버피팅 문제가 감소



◆ Spark ML 모델 튜닝 (Tuning)

❖ **CrossValidator, TrainValidationSplit**

- TrainValidationSplit: 홀드아웃 기반 테스트 수행
- CrossValidator: 교차분석 (K-Fold) 기반 테스트 수행
- 다음과 같은 입력을 기반으로 가장 좋은 파라미터를 찾아줌
 - Estimator (머신러닝 모델 혹은 ML Pipeline)
 - Evaluator: 머신러닝 모델의 성능을 나타내는 지표
 - Parameter (훈련 반복 회수 등의 하이퍼 파라미터)
 - ParamGridBuilder를 이용해 ParamGrid 타입의 변수 생성
- 최종적으로 가장 결과가 좋은 모델을 리턴!

◆ Spark ML 머신러닝 모델 성능 측정

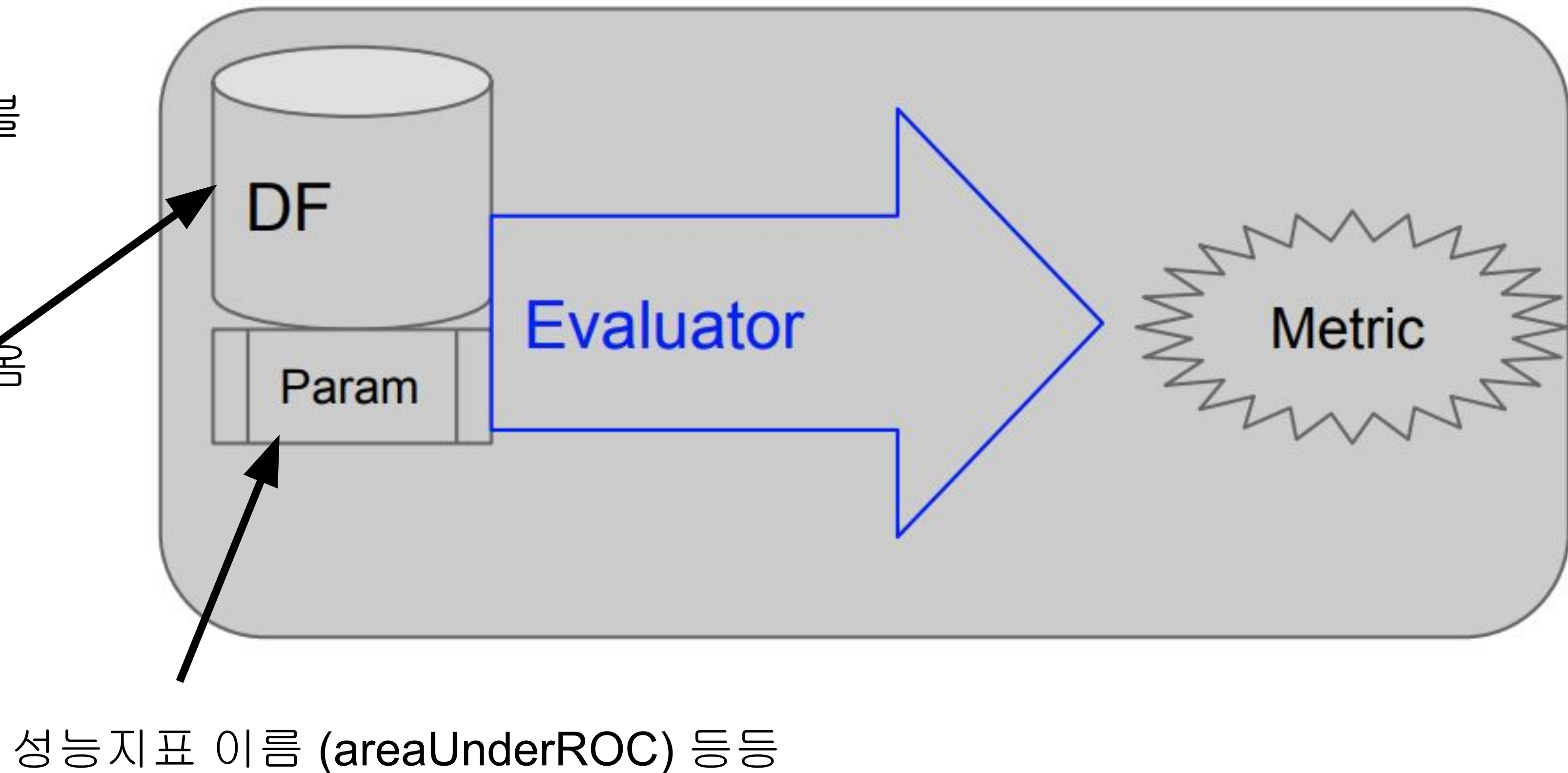
◆ Evaluator

- **evaluate** 함수가 제공됨
 - 인자로 테스트셋의 결과가 들어있는 데이터프레임과 파라미터가 제공
- 머신러닝 알고리즘에 따라 다양한 **Evaluator**가 제공됨
 - RegressionEvaluator
 - BinaryClassificationEvaluator
 - AUC(Area Under the Curve)가 성능지표가 됨
 - MulticlassClassificationEvaluator
 - MultilabelClassificationEvaluator
 - RankingEvaluator

◆ Spark ML 머신러닝 모델 성능 측정

◆ Evaluator

- Prediction 컬럼과 레이블 컬럼이 들어있는 데이터프레임. Logistic Regression의 경우, probability 컬럼도 들어옴
- 보통 이 데이터프레임은 머신러닝 모델의 transform 함수가 리턴해준 값이 됨

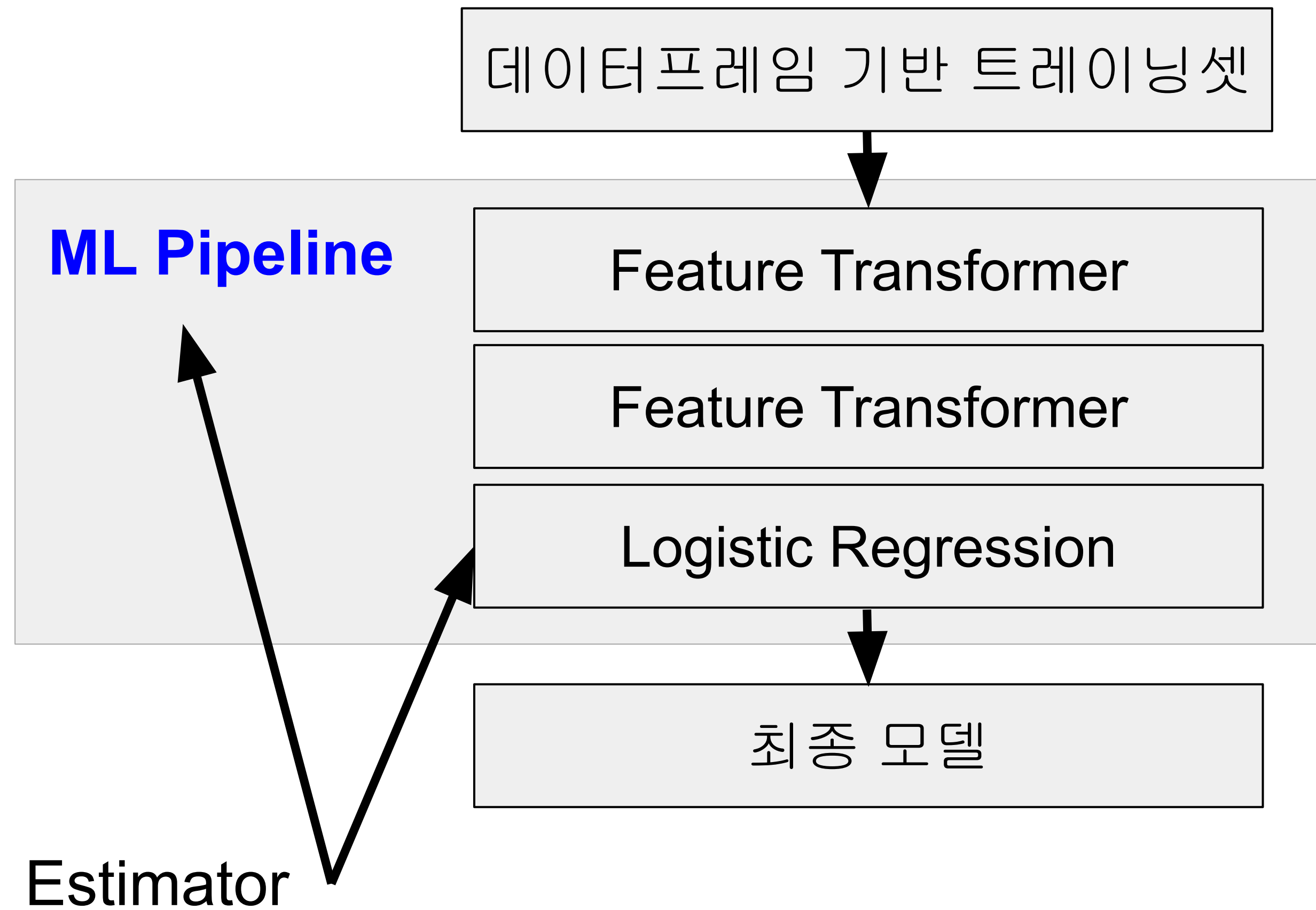


◆ Spark ML 모델 튜닝

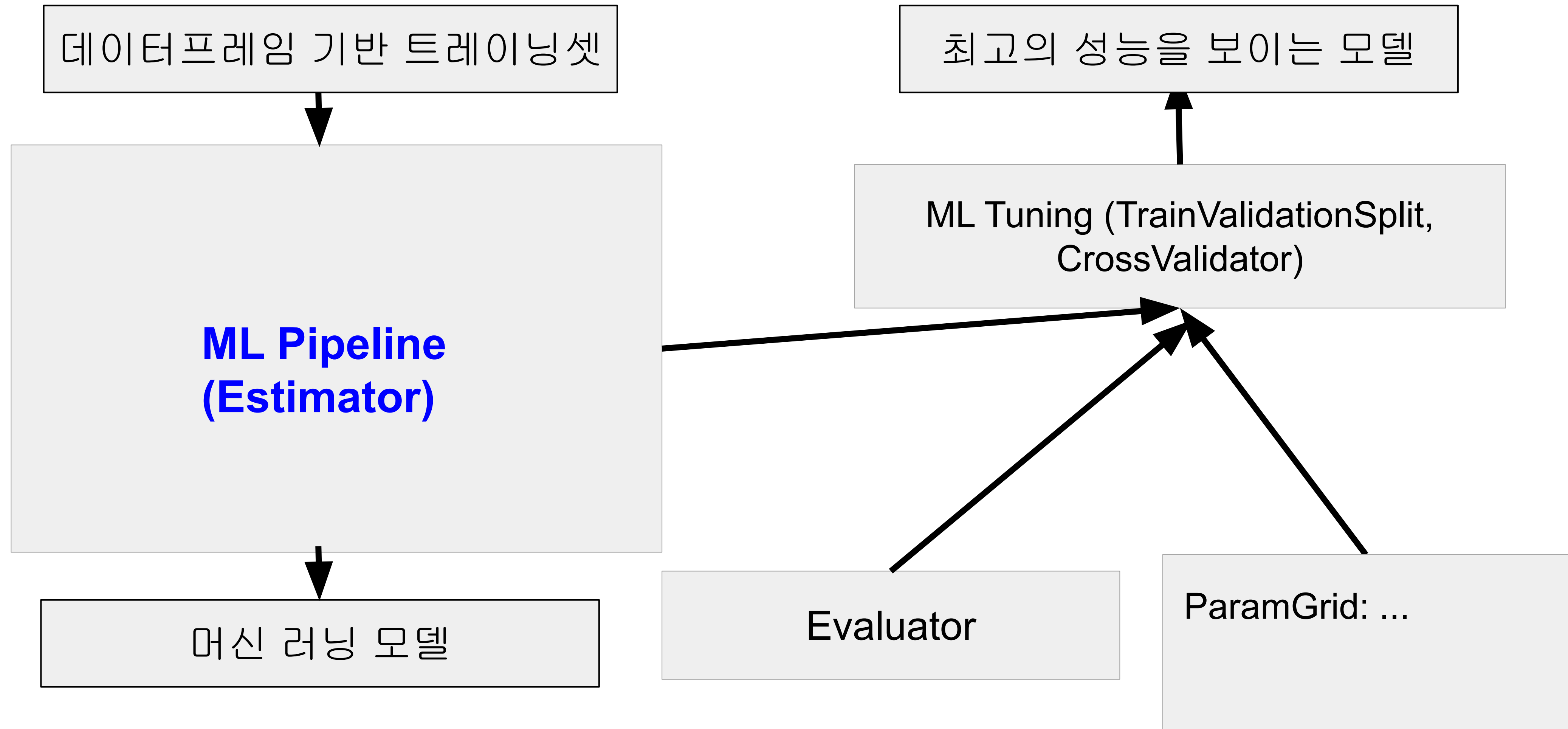
❖ 모델 선택시 입력

- Estimator (앞서 설명)
 - 머신러닝 알고리즘이나 모델 빌딩 파이프라인 (ML Pipeline)
- **Evaluator**: 머신러닝 모델 성능 측정에 사용되는 지표 (metrics)
- 모델의 파라미터 맵
 - 파라미터 그리드(**ParamGrid**)라고 불리기도 하는데 모델 테스트시 고려해야하는 가능한 러닝 관련 파라미터들로 주로 트리 관련 알고리즘에서 중요함
 - 테스트되는 파라미터의 예로는 트리의 최대 깊이, 훈련 횟수 등이 있음

◆ Spark ML 머신러닝 모델 빌딩 전체 프로세스 (1)



◆ Spark ML 머신러닝 모델 빌딩 전체 프로세스 (2)



실습: ML Pipeline 기반 머신러닝 모델 만들기

ML Pipeline을 써서 타이타닉 승객 예측 모델 만들기

◆ 타이타닉 승객 생존 예측 분류기 개발 방향 (1)

- ❖ 이번에는 ML Pipeline을 사용하여 모델을 빌딩
- ❖ 다양한 Transformer 사용
 - Imputer, StringIndexer, VectorAssembler
 - MinMaxScaler를 적용하여 피쳐 값을 0과 1사이로 스케일
- ❖ GBTClassifier와 LogisticRegression을 머신러닝 알고리즘으로 사용
 - Gradient Boosted Tree Classifier
 - 의사결정 트리(Decision Tree)의 머신러닝 알고리즘
 - Regression과 Classification 모두에 사용가능

◆ 타이타닉 승객 생존 예측 분류기 개발 방향 (2)

- ❖ CrossValidation을 사용하여 모델 파라미터 선택
 - BinaryClassificationEvaluator를 Evaluator로 사용
 - ParamGridBuilder를 사용하여 ParamGrid를 생성
 - 뒤에 설명할 ML Pipeline을 인자로 지정
 - ML Pipeline를 만들 때 머신러닝 알고리즘을 마지막에 지정해야 함

◆ MinMaxScaler 사용 (1)

❖ 기본적으로 VectorAssembler로 벡터로 변환된 피처컬럼에 적용

```
from pyspark.ml.feature import MinMaxScaler
```

```
age_scaler = MinMaxScaler(inputCol="features", outputCol="features_scaled")
```

```
age_scaler_model = age_scaler.fit(data_vec)
```

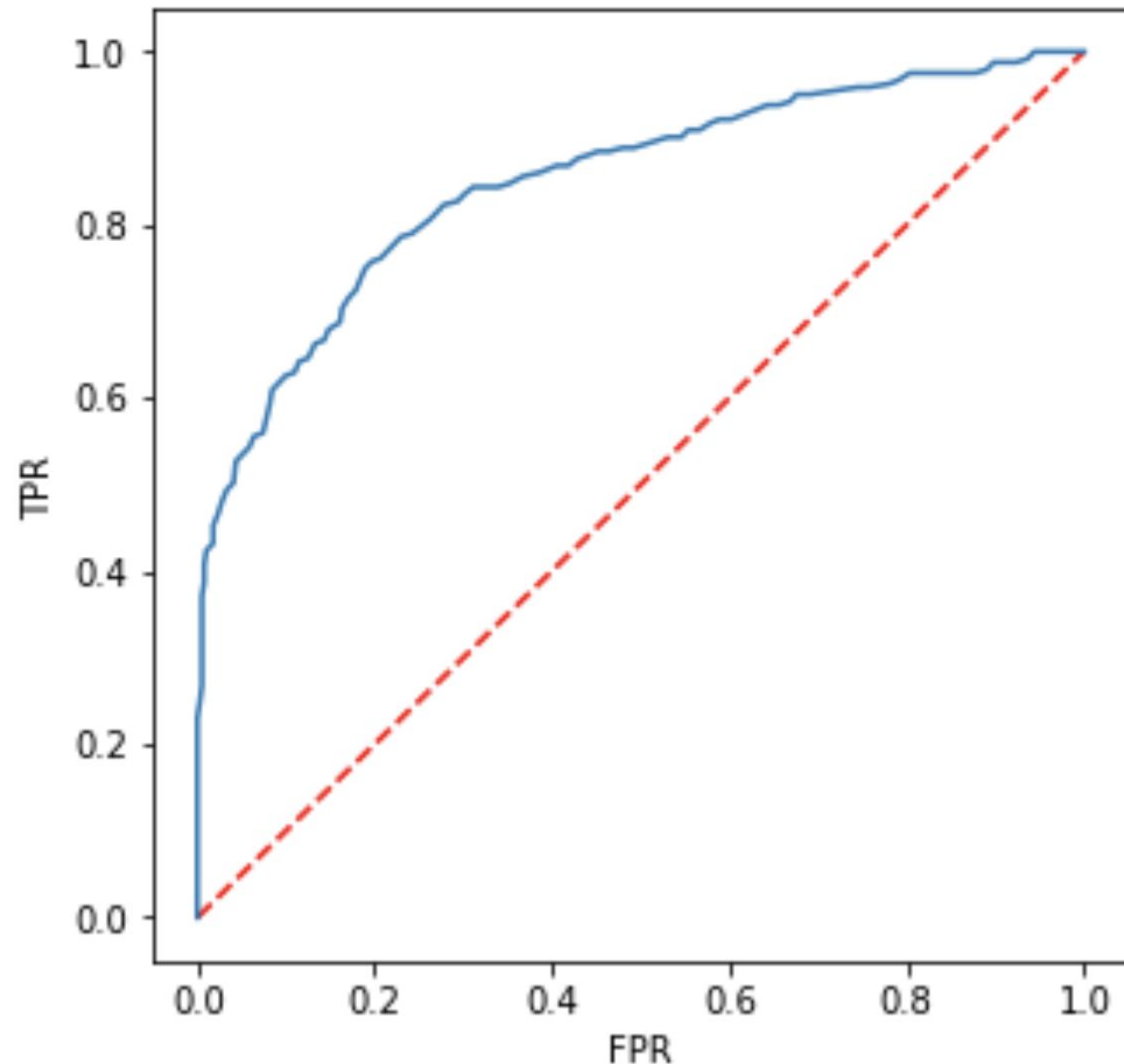
```
data_vec = age_scaler_model.transform(data_vec)
```

```
data_vec.select("features", "features_scaled").show()
```

```
+-----+-----+
|      features      | features_scaled |
+-----+-----+
|[3.0,1.0,0.0,7.25... |[1.0,0.125,0.0,0....|
|[1.0,1.0,0.0,71.2... |[0.0,0.125,0.0,0....|
```

◆ MinMaxScaler 사용 (2)

❖ 이 경우 AUC의 점수가 0.03 정도 올라감



```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(5,5))
```

```
plt.plot([0, 1], [0, 1], 'r--')
```

```
plt.plot(model.summary.roc.select('FPR').collect(),  
         model.summary.roc.select('TPR').collect())
```

```
plt.xlabel('FPR')
```

```
plt.ylabel('TPR')
```

```
plt.show()
```

◆ ML Pipeline 사용 절차

- ❖ 트레이닝셋에 수행해야하는 **feature transformer**들을 생성
- ❖ 사용하고자 하는 머신러닝 모델 알고리즘(**Estimator**)을 생성
- ❖ 이 것들을 순서대로 파이썬 리스트에 추가
 - 머신러닝 알고리즘이 마지막으로 추가되어야 함
- ❖ 이 파이썬 리스트를 인자로 **Pipeline** 개체를 생성
- ❖ 이 **Pipeline** 개체를 이용해 모델 빌딩: 2가지 방법 존재
 - 하나는 이 **Pipeline**의 **fit** 함수를 호출하면서 트레이닝셋 데이터프레임을 지정
 - 다른 하나는 이를 뒤에서 설명한 **ML Tuning** 개체로 지정해서 여러 하이퍼 파라미터를 테스트해보고 가장 결과가 좋은 모델을 선택하는 것!
 - 이 때 보통 교차분석을 사용해 가장 좋은 모델을 선택!

◆ ML Pipeline 사용 예 (1)

❖ 필요한 Transformer와 Estimator들을 만들고 순서대로 리스트에 추가

```
from pyspark.ml.feature import Imputer, StringIndexer, VectorAssembler, MinMaxScaler
```

```
from pyspark.ml.classification import LogisticRegression
```

```
stringIndexer = StringIndexer(inputCol = "Gender", outputCol = 'GenderIndexed')
```

```
imputer = Imputer(strategy='mean', inputCols=['Age'], outputCols=['AgeImputed'])
```

```
assembler = VectorAssembler(inputCols=['Pclass', 'SibSp', 'Parch', 'Fare', 'AgeImputed',  
'GenderIndexed'], outputCol="features")
```

```
minmax_scaler = MinMaxScaler(inputCol="features", outputCol="features_scaled")
```

```
algo = LogisticRegression(featuresCol="features_scaled", labelCol="Survived")
```

```
stages = [stringIndexer, imputer, assembler, minmax_scaler, algo]
```

◆ ML Pipeline 사용 예 (2)

❖ 앞서 만든 리스트를 Pipeline의 인자로 지정!

```
from pyspark.ml import Pipeline
```

```
pipeline = Pipeline(stages = stages)
```

```
# 이를 사용해 바로 모델 빌드를 하고 싶다면
```

```
df = data.select(['Survived', 'Pclass', 'Gender', 'Age', 'SibSp', 'Parch', 'Fare'])
```

```
train, test = data.randomSplit([0.7, 0.3])
```

```
lr_model = pipeline.fit(train)
```

```
lr_cv_predictions = lr_model.transform(test)
```

```
evaluator.evaluate(lr_cv_predictions)
```

◆ ML Tuning 사용 절차

- ❖ 테스트하고 싶은 머신러닝 알고리즘 개체 생성 (혹은 ML Pipeline)
- ❖ ParamGrid를 만들어 테스트하고 싶은 하이퍼 파라미터 지정
- ❖ CrossValidation 혹은 TrainValidationSplit 생성
- ❖ fit 함수 호출해서 최선의 모델 선택

◆ ML Tuning 사용 예 (1)

❖ ParamGrid와 CrossValidator 생성

```
from pyspark.ml.tuning import ParamGridBuilder, CrossValidator
```

```
paramGrid = (ParamGridBuilder()  
             .addGrid(algo.maxIter, [1, 5, 10])  
             .build())
```

```
cv = CrossValidator(  
    estimator=pipeline,  
    estimatorParamMaps=paramGrid,  
    evaluator=evaluator,    # 앞서 생성한  
    numFolds=5  
)
```

◆ ML Tuning 사용 예 (2)

❖ CrossValidation 실행하여 최선의 모델 선택하고 테스트셋으로 검증

Feature transform 전의 트레이닝셋을 7 대 3으로 나눔

```
train, test = data.randomSplit([0.7, 0.3])
```

교차 분석 수행! 가장 좋은 모델이 리턴됨

```
cvModel = cv.fit(train)
```

이 모델에 테스트셋을 입력으로 주고 결과를 받아 결과 분석!

```
lr_cv_predictions = cvModel.transform(test)
```

```
evaluator.evaluate(lr_cv_predictions)
```


◆ ML Tuning 사용 예 (3)

- ❖ 어느 하이퍼 파라미터 조합의 최선의 결과를 냈는지 알고 싶다면?
 - ML Tuning의 `getEstimatorParamMaps/getEvaluator`의 조합으로 파악
 - 앞서 Logistic Regression의 경우 아래와 같은 결과 도출

	areaUnderROC	maxIter
0	0.838198	1
1	0.852535	5
2	0.852942	10

◆ Colab 데모

❖ [PySpark ML: 5일 차](#)

범용 머신러닝 모델 파일포맷: PMML

머신러닝 모델이 만들어졌으면 다음 스텝은?

- ◆ 다양한 머신러닝 개발 플랫폼들이 넘쳐남!
 - ❖ 많은 사람들이 이미 Scikit-Learn, PyTorch, Tensorflow 등을 맛봄
 - 거기에 Spark ML 추가 :)
 - ❖ 이런 환경에서 통용되는 머신러닝 파일포맷이 있다면 어떨까?
 - ❖ 그래서 나온 모듈 혹은 파일포맷등이 있음
 - PMML과 MLeap이 대표적
 - 이게 가능해지면 머신러닝 모델 서빙환경의 통일이 가능
 - 실상은 이런 공통 파일포맷이 지원해주는 기능이 미약해서 복잡한 모델의 경우에는 지원불가

◆ PMML

❖ Predictive Model Markup Language

❖ Machine Learning 모델을 마크업 언어로 표현해주는 XML 언어

- 간단한 입력 데이터 전처리와 후처리도 지원. 하지만 아직도 제약사항이 많음
- Java 기반:
 - <https://github.com/jpmml/jpmml-evaluator>
 - 많은 회사들이 모델 실행을 위해서 자바로 PMML 엔진을 구현
- PySpark에서는 pyspark2pmml를 사용
 - 하지만 내부적으로는 jpmml-sparkml이라는 자바 jar 파일을 사용
 - 너무 복잡함. 버전 의존도도 복잡

◆ 전체적인 절차

1. ML Pipeline을 PMML 파일로 저장

- 이를 위해 pyspark2pmml 파이썬 모듈을 설치
 - [jpmml-sparkml-executable-1.6.3.jar](#) 파일의 설치가 필요
- pyspark2pmml.PMMLBuilder를 이용하여 ML Pipeline을 PMML 파일로 저장

2. PMML 파일을 기반으로 모델 예측 API로 론치

- Openscoring 프레임워크
- AWS SageMaker
- Flask + PyPMML

3. 이 API로 승객정보를 보내고 예측 결과를 받는 클라이언트 코드 작성

◆ 머신 러닝 모델을 PMML 파일로 저장하는 예제

- ❖ cvModel은 머신러닝 모델이나 ML Pipeline

- ❖ train_fr은 트레이닝셋 데이터프레임

```
from pyspark2pmml import PMMLBuilder
```

```
pmmlBuilder = PMMLBuilder(spark.sparkContext, train_fr, cvModel)
```

```
pmmlBuilder.buildFile("Titanic.pmml")
```

◆ PMML 파일을 PyPMML로 로딩하고 호출하는 예제

❖ 로딩 예제 (Model.load)

```
from pypmml import Model  
model = Model.load('single_iris_dectree.pmml')
```

❖ 예측 예제 (predict)

```
model.predict({'sepal_length': 5.1, 'sepal_width': 3.5, 'petal_length': 1.4, 'petal_width': 0.2})
```


요약

머신러닝 모델 튜닝 대해 배움
ML Pipeline과 CrossValidation을 사용해서 모델 빌딩해봄
만들어진 모델을 실제 환경에서 API로 론치하는 방법 설명



Grepp Inc

한기용

keeyonghan@hotmail.com